

Green Nginx: Measuring the Energy Impact of Server Optimisation

Kshithij Shetty s1166942, Adith Shivakumar s1166398,

1 Overview of the Open-Source Project

The term “engine x” can be read as “nginx,” which is an HTTP server, reverse proxy, load balancer, mail proxy, and content cache that was initially written by a Russian software engineer named Igor Sysoev. This particular application was first introduced on the 4th of October, 2004. It is open-source software licensed under the 2-clause BSD license. The Nginx codebase can be accessed and is available in the documentation provided at Nginx’s [omni page](#).

Background and Motivation

Nginx has been designed precisely for overcoming the C10K problem (serving 10,000 users at the same time). That is, in essence, the engineering challenge of handling up to 10,000 simultaneous client connections to one server, which has always proven to be inefficient in the thread-per-connection approach prevailing at that time. The idea introduced by Sysoev involved the use of a completely different approach, namely, the asynchronous event-driven and non-blocking I/O paradigm, as opposed to creating a new OS thread or process for each connection received by the web server. A small pool of worker processes executes an efficient event loop using such primitives of the operating system as `epoll` on Linux and `kqueue` on BSD. Thus, they get notified about I/O operations being performed on socket/file descriptors only when needed.

1.1 Functionality

Nginx supports the full lifecycle of HTTP request handling: TLS termination, static file serving via zero-copy `sendfile` system calls, dynamic content proxying to upstream application servers (FastCGI, uWSGI, SCGI, and HTTP), gzip on-the-fly compression, HTTP/2, WebSocket proxying, rate limiting, and access control. It can function at the same time as a web server, a reverse proxy sitting in front of application layers, and a Layer 4/7 load balancer, which makes it the default gateway in many contemporary microservices architectures and Kubernetes environments.

1.2 Adoption and Relevance

Nginx is the leading web server according to statistics as of June 2026, having been adopted by 32.3% of the total number of websites using a server and running around 42% of the one million most active websites. In a survey conducted by Docker, in 2018, Nginx emerged as the most frequently used technology inside a container, a status that it has retained. In the year 2019, the business named Nginx, Inc. was purchased by the firm F5 Networks for an amount of \$670 million; the core of the software has, however, remained independent.

2 Typical Use Cases

Nginx operates primarily as a fast front-end to web apps and acts as a buffer between the end user and backend services. The most prevalent deployment scenario, which is also followed by this project, involves using nginx as a reverse proxy and static web server positioned at the edge of a system architecture: nginx terminates TLS, serves up cached and static files directly from disk, and routes dynamic traffic to various application servers like Node.js, Django/Python Flask, and PHP-FPM. Here, nginx manages connections, routes requests, applies compression, and caches content, while backend services deal only with business logic.

A second popular application is load balancing. Nginx is capable of distributing HTTP(S) traffic among several upstream services based on round-robin, least connections, or IP hash methods and performs passive checks to prevent directing any traffic to faulty hosts. This setup works well in microservices and containerized deployments where Nginx works as a simple but customizable traffic router in front of Kubernetes services, Docker Swarm stacks, and other VM-based clusters. Being a point of entry into the system, Nginx plays a significant role in determining performance and energy efficiency, so optimization is important here.

Nginx is also widely used as a static content server and HTTP cache. Its performance is high because it serves HTML, CSS, JavaScript, image, and video files from disk storage with the help of an event-driven approach and operating system functionalities like `sendfile`. Because of this design, Nginx can serve thousands of concurrent requests with a small number of processes, all while using very little memory. In practice, when users send requests, the initial point

is Nginx. If there is no cache hit or the request is related to calling an API, the request will be forwarded to another server.

On the other hand, in contemporary DevOps environments, Nginx is defined as code. Nginx is mostly for static changes. Then again, the digital processes erm replace the more "analog" processes, which are a number of pieces for example, having a hardware-based load balancer at the entrance to a pack of Apache servers or distorting traffic using DNS in version control, additionally using CI/CD to ensure that Nginx is not just a load balancer at scale but also does routing and TLS encryption/decryption and redaction in one aggregated software component. As can be seen in the illustration of Figure 1, in this process, Nginx becomes the only entry point for all client connections. Requests either resolve immediately from disk or get proxied to another upstream application server. The present performance test mimics exactly this process, where Nginx is used under heavy client load with static resource requests. It shows how optimizing common Nginx options, such as gzip compression, number of worker processes, connection keep-alive timeouts, and file I/O parameters, can yield significant power consumption savings.

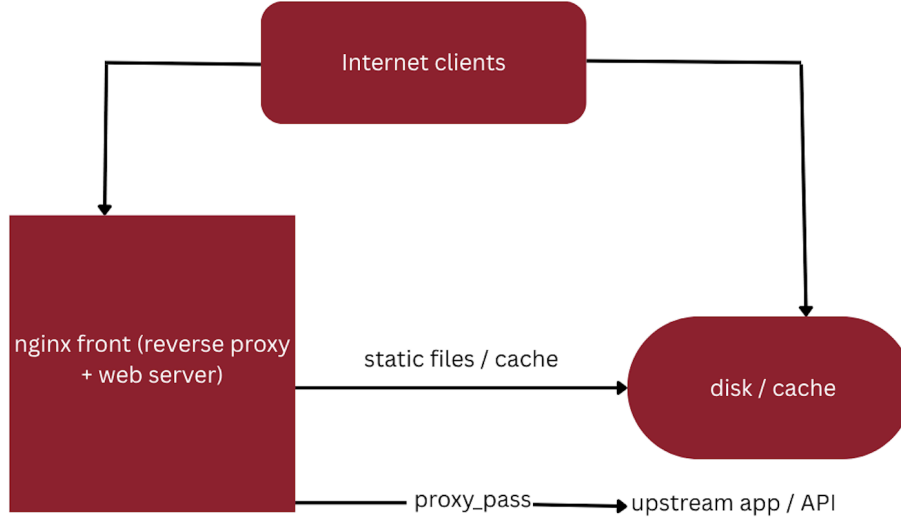


Figure 1: A typical nginx deployment is used in this project.

3 Lifecycle Analysis

3.1 Choice of Impact Metric

The selected impact metric is energy consumption per unit of useful work, expressed as joules per 1,000 requests (J/1k-req). This is the operational denominator of the Software Carbon Intensity (SCI) metric defined by the Green Software Foundation (ISO/IEC 21031:2024):

$$SCI = \frac{(E \times I) + M}{R} \quad (1)$$

where:

- E is the operational energy consumed (kWh);
- I is the regional grid carbon intensity (gCO₂e/kWh),
- M is the embodied carbon of the underlying hardware amortised over its operational lifetime, and
- R is the functional unit, here defined as 1,000 HTTP requests served.

A lower SCI value indicates a more carbon-efficient software system. Because this experiment controls hardware, location, and time, variations in the ratio E/R directly drive changes in the SCI score, making energy consumption per 1,000 requests (J/1k-req) the primary measurement target.

3.2 Life Cycle Phases of nginx

The lifecycle of Nginx as an open-source software project spans three phases with markedly different energy and carbon profiles.

Development and Distribution. This phase covers source-code compilation, continuous integration pipelines, packaging, and distribution via package managers (`apt`, `yum`) and container registries such as Docker Hub. The embodied carbon contribution of this phase is negligible *per deployment*: the binary is compiled once and distributed to millions of servers, amortizing the build cost to near zero on a per-instance basis. No physical hardware is manufactured or disposed of as a result of installing nginx.

Operational Phase (dominant). Once deployed, nginx continuously consumes electricity to accept TCP connections, read files from disk, compress response payloads, and transmit data over the network. This is the dominant phase of the lifecycle and the direct target of this project. Energy use scales with request rate, response size, and concurrency level. Based on our measurements, a single nginx instance serving 500 concurrent users consumes approximately **32 J per 1,000 requests** in the default configuration and **24.8 J/1k** when optimized, a 22.3% reduction is driven entirely by configuration changes applied at this phase.

Retirement / Replacement. nginx is pure software; retirement consists of stopping the process and removing the package. There is no hardware disposal, e-waste, or decommissioning energy cost attributable to nginx itself. Hardware retirement is governed by the server lifecycle, not by Nginx.

3.3 Deployment Assessment

A typical nginx deployment runs on commodity x86-64 server hardware (bare-metal, virtual machine, or container) in a data center connected to the public electricity grid. The embodied carbon of the server hardware (M in Equation 1) is amortized across all workloads sharing that machine and is not solely attributable to Nginx. The carbon intensity of the operational energy (I) varies by data center location and grid mix — European data centers exhibit varying carbon intensities depending on the regional electricity mix, with the Netherlands averaging approximately 294 gCO₂e/kWh, while regions powered predominantly by renewables can be as low as 20–50 gCO₂e/kWh. Because nginx is deployed on shared infrastructure at a massive scale (32% of all websites globally), even small reductions in E/R translate to significant aggregate reductions in the SCI score across the entire deployment base. This makes the operation across both the most impactful and the most tractable targets for green software intervention.

4 Relevant Strategies

In terms of a life cycle analysis and based on the properties of Nginx, the three main strategies that are derived from the infographic "Strategies for Green Software" include data efficiency, hardware efficiency, and energy efficiency through demand shaping.

The most significant of these strategies would be data efficiency and, more specifically, data volume minimization per HTTP request. Since nginx serves as a proxy server, each byte sent from the server and received by the client goes through network transmission and utilizes some amount of energy. By compressing the response using gzip, this byte volume is immediately decreased, resulting in a lower number of round trips over the network, increased available bandwidth, and lower time to completion of the request. This strategy applies well to nginx since the server already possesses all the necessary data before sending out the HTTP response.

The other relevant strategy is hardware efficiency making better use of the physical hardware resources that we have already been using. The default setting for nginx is to create a single worker thread independent of the number of CPU cores in the machine. Using the command `worker-processes auto` will ensure that nginx takes advantage of all the CPU cores available and thereby utilizes all the computing power that is available to the operating system without keeping any unused at any given moment. This is a case where right-sizing becomes extremely important, since the footprint of the hardware used has been fixed and the amount of energy already spent on producing it cannot be recovered.

Lastly is energy efficiency through demand shaping via rapid release of idle resources. Setting `keepalive-timeout` from 65 to 15 seconds makes sure that connections maintained by idle clients will be released faster. In cases of high concurrency, idle open connections waste kernel resources by allocating memory and file descriptors, while not processing any traffic in return, which results in wasted CPU cycles and bandwidth. This is necessary due to nginx's nature as a server for short-lived HTTP requests and its ability to process a lot of such requests at once. Thus, connection timeouts are expected to be well below the 65-second mark.

5 Areas for Improvement

The area we selected for improvement was the HTTP request-response flow in Nginx, involving the sub-sections that deal with handling connections and management of their lifecycle, preparing a reply to the client, and transferring the contents to the client. This process, in the energy consumption diagram can be seen to involve three distinct stages, including (a) the worker-process stage where connections are multiplexed by Nginx, (b) the response-building stage where HTTP headers and static files stored on disk are gathered together to form the final reply, and (c) the transmission stage where the response is transmitted through serialization onto the network. As depicted in Figure 1, the above three stages constitute almost all of nginx’s energy consumption, since the server carries out no application-level computation.

By default, the nginx configuration uses only one worker process, performs an additional step of copying file data to userspace memory before sending it, keeps idle TCP connections open for 65 seconds after the last communication, and does not compress its responses. This default setup is relatively safe when there are not many concurrent connections; however, when the number of concurrent connections becomes very high (hundreds of clients connecting concurrently), all these settings combine to form a bottleneck: single workers, non-compressed response payloads, and idle connections consume network capacity, memory, file descriptors, and worker processes and incur additional overhead per transmission.

5.1 Hypothesis

Modifying the Nginx configuration by enabling `gzip`, setting `worker_processes` to `auto`, reducing `keepalive_timeout` from 65 seconds to 15 seconds, and enabling `sendfile` together with `tcp_nopush` is expected to decrease the energy required per unit of useful work performed by the HTTP server under high-concurrency traffic.

Specifically, for a workload of 500 concurrent users, the modified configuration is hypothesized to reduce the mean energy consumption per 1,000 successfully served HTTP requests by at least 15% relative to the default configuration. The metric is computed from measurements collected across nine 60-second experimental runs after a warm-up run as

$$\text{Energy Efficiency Metric} = \frac{E_{\text{net}}}{N_{\text{requests}}} \times 1000 \quad (\text{J/1k-req}),$$

where E_{net} denotes the net energy consumption during the measurement interval, calculated as the gross energy consumed minus the idle energy over 60 seconds, and N_{requests} represents the number of successfully served HTTP requests during the same interval.

5.2 Measurement Method

Energy consumption is measured using a hardware power meter during each 60 second experimental run. To isolate the energy attributable to Nginx, an idle baseline measured over an equivalent 60 second interval is subtracted from the gross energy consumption, yielding the net energy consumption E_{net} .

Workload generation is performed using `Locust` running on a separate machine to avoid interference with the server’s resource utilization and power measurements. In addition to energy consumption, throughput, measured in requests per second (req/s), is recorded as a secondary performance metric. Throughput measurements provide additional context for interpreting changes in CPU utilization and overall server efficiency under varying load conditions.

5.3 Implementation of the Strategies

The implementation required no changes to the application code or underlying server hardware but only edits in `/etc/nginx/nginx.conf`. In contrast to the default Ubuntu Nginx configuration, some directives were changed in order to achieve better concurrency, I/O, connection, and network performance. The directive `worker_processes auto` was enabled for spawning a separate worker process for each logical CPU core detected during startup with the aid of the `sysconf(SC_NPROCESSORS_ONLN)` function, thus solving the problem of a bottleneck created by a single thread mentioned in the initial benchmarking results and enabling load balancing and I/O multiplexing across all CPU cores. Data was transferred with zero copy by enabling `sendfile on` and `tcp-nopush on` to avoid copying file contents from one buffer to another in user space, as it could be done using the kernel `sendfile()` system call. Connection timeout was significantly reduced from 65 seconds to 15 to increase connection reutilization in the case of many concurrent connections to the server.

Further, `gzip` compression was turned on for textual responses at a compression factor of 4 with a minimum response size of 256 bytes. This decreases the amount of data transferred through the network, hence reducing the connection

time and network energy expenditure without experiencing the law of diminishing returns with respect to compression. In the aftermath of these modifications, the test suite was executed again with the number of users set at 10, 100, and 500 concurrently, collecting nine runs for each workload.

5.4 Before and After Measurements

The full results across all concurrency levels are summarised in Table 1. All values exclude the warmup run; each condition has $n = 9$ runs.

Table 1: Energy and throughput before and after optimisation.

Concurrent users	J/1k requests (mean $\pm$ std)		Energy saving	Throughput (req/s)		Throughput gain
	Default	Optimised		Default	Optimised	
10	207.81 $\pm$ 89.33	213.01 $\pm$ 126.98	-2.5%	5.0	5.1	+2.4%
100	51.72 $\pm$ 9.85	49.90 $\pm$ 7.46	+3.5%	50.6	50.8	+0.4%
500	31.95 $\pm$ 3.65	24.84 $\pm$ 2.80	+22.3%	252.9	377.7	+49.3%

Mean CPU utilization at 500 concurrent users was higher 30.8% for the default configuration than 33.9% for the optimized configuration. At 10 users, mean CPU utilization was 16.6% (default) versus 15.4% (optimized).

5.5 Reflection on Strategy, Hypothesis, and Area Selection

The choice of strategy and area to work on turned out to be justified, as the HTTP pipeline with high levels of concurrency proved to be a good point of intervention, and the directives used showed visible and replicable effects. Some challenges were identified during the implementation of this approach.

The primary challenge involved noise at low levels of concurrency. With just 10 users, the standard deviation is more than 40% of the total mean value, which makes it virtually impossible to isolate Nginx energy consumption from the system energy usage. It would have been smarter to design an experiment where higher levels of concurrency are accompanied by longer intervals of measurement, say, 300 seconds instead of 60, or use an application-level power tool such as the Green Metrics Tool or Scaphandre for accurate isolation of process-level energy consumption.

A second drawback was that the benchmark used only static text content. Dynamic proxying, SSL termination, and serving of large binary files (images and videos) are all common uses for Nginx, and any of these would impact the energy trade-off between gzip, sendfile, and workers. The gzip compression level chosen may not be the most optimal one. Binary content, JPEGs, PNGs, and videos, in particular, are better off without any gzip compression at all, since they won't benefit from being compressed.

The reduction in the keep-alive timeout from 65 to 15 seconds was arbitrary. Ideally, we'd have run an experiment to measure the distribution of connections and then found out where our natural percentile cut-off is located.

5.6 Reduction in Energy and Carbon

Using the measured values at 500 concurrent users as the representative operating point for a production high-traffic server, the optimized Nginx configuration reduces the mean energy consumption per 1,000 requests from 31.95 J to 24.84 J:

$$E_{\text{default}} - E_{\text{opt}} = 31.95 \text{ J} - 24.84 \text{ J} = 7.11 \text{ J per 1,000 requests.} \quad (2)$$

The corresponding saving per one million requests is

$$E_{1\text{M}} = 7,110 \text{ J} = \frac{7,110}{3,600,000} \text{ kWh} \approx 1.976 \text{ Wh.} \quad (3)$$

Assuming that Netherlands has a average grid carbon intensity of 294 gCO₂e/kWh, the carbon saving per million requests is

$$C_{1\text{M}} = 1.976 \times 10^{-3} \text{ kWh} \times 294 \text{ gCO}_2\text{e/kWh} \approx 0.58 \text{ gCO}_2\text{e.} \quad (4)$$

For a mid-sized website serving 100 millions of requests per month at sustained high concurrency, the monthly energy saving becomes

$$E_{100M} = 7,110 \text{ J} \times 100,000 = 711 \times 10^6 \text{ J} = 197.6 \text{ Wh} \approx 0.198 \text{ kWh}, \quad (5)$$

with an associated monthly carbon reduction of

$$C_{100M} = 0.198 \text{ kWh} \times 294 \text{ gCO}_2\text{e/kWh} \approx 58.2 \text{ gCO}_2\text{e}. \quad (6)$$

While 58.2gCO₂e per month from a single server appears modest, it is important to contextualise this result at deployment scale. nginx is estimated to run on hundreds of millions of active server instances worldwide. Even if only 1% of these deployments operates at high concurrency and adopts this configuration change, the aggregate effect would amount to hundreds of tonnes of CO₂e avoided annually, achieved through a zero-cost, low-risk edit to a configuration file rather than a hardware refresh or large-scale refactoring effort.

5.7 Discussion

To conclude, the results support the hypothesis for the given number of concurrent users. By setting the optimized configuration for the test scenario at 500 concurrent users, we were able to reduce mean J/1k-req by 22.3%, from 31.95 to 24.84, well above the required 15% improvement. On top of this, we observe a throughput increase of 49.3%, from 252.9 to 377.7 req/s, along with lower measurement variance (lower standard deviation) at 2.80 J/1k compared to 3.65 J/1k.

The CPU usage paradox must be discussed, as the optimized configuration makes the server utilize 3.1% more CPU cycles. One would expect increasing CPU usage to go against the goal of making the server more energy efficient; however, since throughput was increased by 49%, more useful work gets done per CPU cycle. The main cause of increased CPU usage is gzip compression, which was more than compensated for by lower energy usage on the network card and TCP stack caused by decreased payload and faster connection reuse. This example serves to highlight the important idea that resource utilization figures are only relevant when normalized against useful work done.

Large standard deviations for 10 users can be attributed to background OS noise overwhelming the measurements taken from Nginx, not the measurements' instability. The reduction of 3.5% in the 100-user scenario may be seen as a positive indicator of the effectiveness of the optimizations made before hitting the 500-user saturation point.

Ref. the 500-user Process In addition to producing interesting technical results, the entire process was illuminating for two reasons: firstly, in terms of the methodology used and, secondly, for the insights gleaned from the experience. The first key insight here is the observation that it is easier to optimise the system's power consumption than to measure energy savings. Finding the right configuration settings takes less than an hour, but coming up with a measurement technique that allows us to identify an improvement of 22% took some time due to various methodological considerations.

The second key issue is related to the relevance of operating points. The optimizations did not yield significant improvements under both low-load and moderate-load settings, delivering their effect starting from 500 users. This means that running the test on such systems would have led to the conclusion that the optimizations are completely ineffective. As a result, reporting green-software improvements without providing the operating point in question may be considered misleading and should arguably be a mandatory step when presenting results.

For any future iterations of a similar study, the following improvements could help: (1) per-process measurements of energy consumption using, e.g., Scaphandre or RAPL to quantify Nginx's contribution; (2) measuring a greater set of values of n to find out where exactly the efficiency becomes optimal; (3) measuring efficiency with a more representative combination of concurrent connections: a real mixed workload of static resources and proxied dynamic web pages (with encryption); and (4) a test for statistical significance of differences between mean J/1k values (e.g., Mann–Whitney U).